

TP FINAL - Sécurisation de Code Vulnérable

Contenu du dossier

Ce TP contient les fichiers suivants :

- `vulnerable.c` - Code vulnérable à analyser et corriger
- `CONSIGNES.md` - Instructions complètes du TP (4 heures)
- `RAPPORT_TEMPLATE.md` - Template à remplir pour votre rapport
- `Makefile` - Pour compiler et tester vos programmes
- `README.md` - Ce fichier

Objectif

Transformer le programme `vulnerable.c` (intentionnellement vulnérable) en code robuste et sécurisé.

Démarrage rapide

1. Vérifier les dépendances

```
make check-deps
```

2. Compiler et tester le code vulnérable

```
make  
./vulnerable
```

3. Créer votre version sécurisée

```
cp vulnerable.c secure.c  
# Éditez secure.c pour corriger les vulnérabilités
```

4. Compiler votre version sécurisée

```
make secure
```

5. Tester avec Valgrind

```
make valgrind
```

Documentation

Lisez attentivement **CONSIGNES.md** pour :

- Les objectifs pédagogiques
- La méthodologie d'analyse
- Les critères d'évaluation
- Les conseils de correction

Livrables

À la fin du TP, vous devez rendre :

1. **secure.c** - Votre code corrigé
2. **RAPPORT.md** - Votre rapport complété (utilisez RAPPORT_TEMPLATE.md)
3. **Makefile** - (déjà fourni, peut être modifié si besoin)

Commandes utiles

```
# Aide complète
make help

# Compiler version vulnérable
make

# Compiler version sécurisée
make secure

# Tests
make test-secure

# Analyse mémoire
make valgrind

# Nettoyer
make clean
```

Types de vulnérabilités à chercher

- Buffer overflows (gets, scanf, strcpy, strcat, sprintf)
- Validation des entrées manquante
- Injections (commandes, path traversal)
- Format string vulnerabilities
- Gestion mémoire incorrecte (fuites, malloc non vérifié)
- Cryptographie faible ou absente

- Backdoors et mots de passe en dur
- Problèmes de logique métier

Ressources

- `man fgets` - Alternative sécurisée à `gets()`
- `man snprintf` - Alternative sécurisée à `sprintf()`
- `man strncpy` - Alternative sécurisée à `strcpy()`
- OpenSSL SHA-256 documentation
- OWASP Top 10

Planning suggéré

- **1h30** : Analyse et identification des vulnérabilités
- **2h00** : Corrections du code
- **0h30** : Documentation et tests finaux

Rappels importants

-  Compilez avec `-Wall -Wextra`
-  Testez avec `valgrind` (0 fuite mémoire attendue)
-  Documentez CHAQUE correction dans le rapport
-  Testez avec des entrées malicieuses
-  Utilisez SHA-256 pour le hashage (pas d'algo maison)

En cas de problème

1. Vérifiez que OpenSSL est installé : `ldconfig -p | grep crypto`
2. Si problème de compilation : `make clean && make secure`
3. Si Valgrind n'est pas installé : `sudo apt install valgrind` (sur Debian/Ubuntu)

License et usage

Ce TP est destiné à un usage pédagogique uniquement. Le code vulnérable ne doit JAMAIS être utilisé en production.

Bon courage pour ce TP final ! 🎓

Pour toute question, référez-vous aux `CONSIGNES.md`.